1. Two Sum

Program: def two_sum(nums, target):
seen = {}
for i, num in enumerate(nums):
diff = target - num
if diff in seen:
return [seen[diff], i]
seen[num] = i

print(two_sum([2,7,11,15], 9))

Output:
[0, 1]

2. Longest Substring Without Repeating Characters

Program: def longest_substring(s):
char_set = set()
left = max_len = 0

```
for right in range(len(s)):
    while s[right] in char_set:
        char_set.remove(s[left])
        left += 1
    char_set.add(s[right])
    max_len = max(max_len, right - left + 1)

return max_len
```

print(longest_substring("abcabcbb"))

Output:
3

3. Merge Intervals

Program: def merge_intervals(intervals):
intervals.sort()
merged = [intervals[0]]

```
for i in intervals[1:]:
    if i[0] <= merged[-1][1]:
        merged[-1][1] = max(merged[-1][1], i[1])
    else:
        merged.append(i)

return merged
```

print(merge_intervals([[1,3],[2,6],[8,10]]))

Output:
[[1, 6], [8, 10]]

4. Product of Array Except Self

Program: def product_except_self(nums):
res = [1] * len(nums)

```
left = 1
for i in range(len(nums)):
    res[i] = left
    left *= nums[i]

right = 1
for i in range(len(nums)-1, -1, -1):
    res[i] *= right
    right *= nums[i]

return res
```

print(product_except_self([1,2,3,4]))

Output:
[24, 12, 8, 6]

5. Valid Parentheses

Program: def is_valid(s):
stack = []
mapping = {')':'(', '}':'{', ']':'['}

```
for char in s:
    if char in mapping:
        top = stack.pop() if stack else '#'
        if mapping[char] != top:
            return False
    else:
        stack.append(char)

return not stack
```

print(is_valid("({[]})"))

Output:
True

6. Kth Largest Element

Program: import heapq

```
def kth_largest(nums, k):
return heapq.nlargest(k, nums)[-1]

print(kth_largest([3,2,1,5,6,4], 2))
```

Output:
5

7. Search in Rotated Sorted Array

Program: def search(nums, target):
left, right = 0, len(nums)-1

```
while left <= right:
    mid = (left + right)//2
    if nums[mid] == target:
        return mid

    if nums[left] <= nums[mid]:
        if nums[left] <= target < nums[mid]:
            right = mid - 1
        else:
            left = mid + 1
    else:
        if nums[mid] < target <= nums[right]:
            left = mid + 1
        else:
            right = mid - 1

return -1
```

print(search([4,5,6,7,0,1,2], 0))

Output:
4

8. Group Anagrams

Program: from collections import defaultdict

def group_anagrams(strs):
anagrams = defaultdict(list)

```
for word in strs:
    key = ''.join(sorted(word))
    anagrams[key].append(word)

return list(anagrams.values())
```

print(group_anagrams(["eat","tea","tan","ate","nat","bat"]))

Output:
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]

9. Sliding Window Maximum

Program: from collections import deque

```
def max_sliding_window(nums, k):
dq = deque()
res = []

for i in range(len(nums)):
    while dq and dq[0] <= i-k:
        dq.popleft()

    while dq and nums[dq[-1]] < nums[i]:
        dq.pop()

    dq.append(i)

    if i >= k-1:
        res.append(nums[dq[0]])

return res
```

```
print(max_sliding_window([1,3,-1,-3,5,3,6,7], 3))
```

Output:
[3, 3, 5, 5, 6, 7]

10. Minimum Platform Problem

Program: def min_platforms(arr, dep):
arr.sort()
dep.sort()

```
i = j = platforms = max_platforms = 0

while i < len(arr) and j < len(dep):
    if arr[i] <= dep[j]:
        platforms += 1
        i += 1
    else:
        platforms -= 1
        j += 1

    max_platforms = max(max_platforms,
platforms)

return max_platforms
```

print(min_platforms([900,940,950,1100],
[910,1200,1120,1130]))

Output:
3

11. Detect Cycle in Linked List

Program: class ListNode:
def **init**(self, val):
self.val = val
self.next = None

def has_cycle(head):
slow = fast = head

```
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next

    if slow == fast:
        return True

return False
```

Output:
(No output unless linked list is created)

12. Top K Frequent Elements

Program: from collections import Counter
import heapq

```
def top_k_frequent(nums, k):
count = Counter(nums)
return heapq.nlargest(k, count.keys(), key=count.get)

print(top_k_frequent([1,1,1,2,2,3], 2))
```

Output:
[1, 2]

13. Word Break

Program: def word_break(s, wordDict):
dp = [False]*(len(s)+1)
dp[0] = True

```
for i in range(1, len(s)+1):
    for j in range(i):
        if dp[j] and s[j:i] in wordDict:
            dp[i] = True
            break

return dp[-1]
```

print(word_break("applepenapple", ["apple","pen"]))

Output:
True

14. Longest Palindromic Substring

Program: def longest_palindrome(s):
res = ""

```
for i in range(len(s)):
    for l, r in [(i,i), (i,i+1)]:
        while l >= 0 and r < len(s) and s[l] == s[r]:
            if (r-l+1) > len(res):
                res = s[l:r+1]
            l -= 1
            r += 1

return res
```

print(longest_palindrome("babad"))

Output:
bab

15. Trapping Rain Water

Program: def trap(height):
left, right = 0, len(height)-1
left_max = right_max = water = 0

```
while left < right:
    if height[left] < height[right]:
        if height[left] >= left_max:
            left_max = height[left]
        else:
            water += left_max - height[left]
        left += 1
    else:
        if height[right] >= right_max:
            right_max = height[right]
        else:
            water += right_max - height[right]
        right -= 1

return water
```

print(trap([0,1,0,2,1,0,1,3,2,1,2,1]))

Output:
6